

```
!pip -q install -U transformers datasets accelerate peft trl bitsandbytes huggingface_hub
```

```

===== 10.7/10.7 MB 109.7 MB/s eta 0:00:00
===== 520.7/520.7 kB 47.2 MB/s eta 0:00:00
===== 383.7/383.7 kB 38.3 MB/s eta 0:00:00
===== 528.8/528.8 kB 47.1 MB/s eta 0:00:00
===== 60.7/60.7 MB 13.5 MB/s eta 0:00:00
===== 612.9/612.9 kB 53.9 MB/s eta 0:00:00
===== 4.2/4.2 MB 131.8 MB/s eta 0:00:00
===== 47.6/47.6 MB 18.7 MB/s eta 0:00:00

```

```

import gc
import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from peft import PeftConfig, PeftModel
from trl import DPOTrainer, DPOConfig

```

```

!nvidia-smi
import torch
print("CUDA:", torch.cuda.is_available())
print("GPU:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else None)

```

Sun Mar 8 11:26:04 2026

```

+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |                       |
+-----+-----+-----+-----+-----+-----+
|   0   Tesla T4              Off        | 00000000:00:04:0  Off |                    0 |
| N/A   50C    P8             9W / 70W   |  3MiB / 15360MiB |      0%   Default   |
+-----+-----+-----+-----+-----+-----+

```

```

+-----+
| Processes:                         GPU Memory |
| GPU  GI    CI           PID  Type   Process name                  Usage |
|      ID    ID                                   |          |
+-----+-----+-----+-----+-----+
| No running processes found              |          |
+-----+
CUDA: True
GPU: Tesla T4

```

Task 1. Dataset Preparation

```
from datasets import load_dataset
```

```
dataset = load_dataset("jondurbin/truthy-dpo-v0.1")
dataset
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

```

```

README.md: 100%                               904/904 [00:00<00:00, 90.9kB/s]

truthy-dpo.parquet: 100%                       653k/653k [00:00<00:00, 3.27MB/s]

Generating train split: 100%                   1016/1016 [00:00<00:00, 29286.25 examples/s]

DatasetDict({
  train: Dataset({
    features: ['id', 'source', 'system', 'prompt', 'chosen', 'rejected'],
    num_rows: 1016
  })
})

```

```

print(dataset["train"].column_names)
print(dataset["train"][0])

```

```
[ 'id', 'source', 'system', 'prompt', 'chosen', 'rejected' ]
{ 'id': '04c275bf738fd391b7fe25e25fe7bed3', 'source': 'truthy_dpo', 'system': 'You are an unbiased, uncensored, helpful assistant'
```

```
print(dataset)
```

```
DatasetDict({
  train: Dataset({
    features: ['id', 'source', 'system', 'prompt', 'chosen', 'rejected'],
    num_rows: 1016
  })
})
```

```
split_dataset = dataset["train"].train_test_split(test_size=0.2, seed=42)
```

```
train_data = split_dataset["train"]
eval_data = split_dataset["test"]
```

```
print("Full train:", len(train_data))
print("Full eval:", len(eval_data))
```

```
Full train: 812
Full eval: 204
```

Create subset for fast training

Task 2 — Train a Model with DPOTrainer

choose model and load tokenizer

```
model_name = "Qwen/Qwen2.5-1.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
tokenizer.pad_token = tokenizer.eos_token

print("Tokenizer loaded")
print("Pad token:", tokenizer.pad_token)
print("Pad token id:", tokenizer.pad_token_id)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en

```
config.json: 100% 660/660 [00:00<00:00, 36.4kB/s]
```

```
tokenizer_config.json: 7.30k/? [00:00<00:00, 324kB/s]
```

```
vocab.json: 2.78M/? [00:00<00:00, 42.0MB/s]
```

```
merges.txt: 1.67M/? [00:00<00:00, 43.1MB/s]
```

```
tokenizer.json: 7.03M/? [00:00<00:00, 100MB/s]
```

```
Tokenizer loaded
Pad token: <|im_end|>
Pad token id: 151645
```

load the base model

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
)

base_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    low_cpu_mem_usage=True,
    dtype=torch.float16,
    quantization_config=bnb_config,
    device_map="auto",
)

base_model.resize_token_embeddings(len(tokenizer))
base_model.config.pad_token_id = tokenizer.pad_token_id
```

```
base_model.config.use_cache = False

print("Base model loaded")
```

model.safetensors: 100%	3.09G/3.09G [00:40<00:00, 65.9MB/s]
Loading weights: 100%	338/338 [00:10<00:00, 32.05it/s]
generation_config.json: 100%	242/242 [00:00<00:00, 23.9kB/s]
Base model loaded	

reference model loading

```
ref_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    low_cpu_mem_usage=True,
    dtype=torch.float16,
    quantization_config=bnb_config,
    device_map="auto",
).eval()

print("Reference model loaded")
```

Loading weights: 100%	338/338 [00:13<00:00, 26.87it/s]
Reference model loaded	

add LoRA

```
peft_config = LoraConfig(
    lora_alpha=128,
    lora_dropout=0.05,
    r=64,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj"],
)

dpo_model = get_peft_model(base_model, peft_config)
dpo_model.print_trainable_parameters()
```

trainable params: 11,927,552 || all params: 1,555,225,600 || trainable%: 0.7669

```
for name, p in dpo_model.named_parameters():
    if p.requires_grad:
        p.data = p.data.to(torch.float16)

from collections import Counter
trainable_dtypes = Counter()
for name, p in dpo_model.named_parameters():
    if p.requires_grad:
        trainable_dtypes[str(p.dtype)] += 1

print(trainable_dtypes)
```

Counter({'torch.float16': 168})

define DPO training config

```
import inspect
from trl import DPOConfig

print(inspect.signature(DPOConfig.__init__))
```

```
(self, output_dir: str | None = None, per_device_train_batch_size: int = 8, num_train_epochs: float = 3.0, max_steps: int = -1,
```

```
training_args = DPOConfig(
    output_dir="./dpo-output",
    num_train_epochs=1,
    learning_rate=5e-7,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
```

```
lr_scheduler_type="linear",
warmup_ratio=0.1,
logging_steps=10,
save_steps=100,
eval_steps=50,
eval_strategy="steps",
gradient_checkpointing=True,
fp16=True,
bf16=False,
remove_unused_columns=False,
report_to="none",
seed=42,
max_length=512,
beta=0.1,
)
```

warmup_ratio is deprecated and will be removed in v5.2. Use `warmup\_steps` instead.

```
from collections import Counter

trainable_dtypes = Counter()
for name, p in dpo_model.named_parameters():
    if p.requires_grad:
        trainable_dtypes[str(p.dtype)] += 1

print(trainable_dtypes)
```

```
Counter({'torch.float16': 168})
```

```
count = 0
for name, p in dpo_model.named_parameters():
    if p.requires_grad:
        print(name, p.dtype)
        count += 1
        if count == 20:
            break
```

```
base_model.model.model.layers.0.self_attn.q_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.0.self_attn.q_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.0.self_attn.k_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.0.self_attn.k_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.0.self_attn.v_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.0.self_attn.v_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.1.self_attn.q_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.1.self_attn.q_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.1.self_attn.k_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.1.self_attn.k_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.1.self_attn.v_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.1.self_attn.v_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.2.self_attn.q_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.2.self_attn.q_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.2.self_attn.k_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.2.self_attn.k_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.2.self_attn.v_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.2.self_attn.v_proj.lora_B.default.weight torch.float16
base_model.model.model.layers.3.self_attn.q_proj.lora_A.default.weight torch.float16
base_model.model.model.layers.3.self_attn.q_proj.lora_B.default.weight torch.float16
```

```
for name, p in dpo_model.named_parameters():
    if p.requires_grad:
        p.data = p.data.to(torch.float16)

print("Converted trainable params to float16")
```

```
Converted trainable params to float16
```

```
from collections import Counter

trainable_dtypes = Counter()
for name, p in dpo_model.named_parameters():
    if p.requires_grad:
        trainable_dtypes[str(p.dtype)] += 1

print(trainable_dtypes)
```

```
Counter({'torch.float16': 168})
```

```

trainer = DPOTrainer(
    model=dpo_model,
    ref_model=ref_model,
    args=training_args,
    train_dataset=train_data,
    eval_dataset=eval_data,
    processing_class=tokenizer,
)

```

Adding EOS to train dataset: 100%

812/812 [00:00<00:00, 7227.14 examples/s]

Tokenizing train dataset: 100%

812/812 [00:02<00:00, 401.49 examples/s]

WARNING:trl.trainer.dpo_trainer:[RANK 0] Mismatch between tokenized prompt and the start of tokenized prompt+rejected. This may

Adding EOS to eval dataset: 100%

204/204 [00:00<00:00, 1566.91 examples/s]

Tokenizing eval dataset: 100%

204/204 [00:01<00:00, 227.09 examples/s]

```

for name, p in trainer.model.named_parameters():
    if p.requires_grad:
        p.data = p.data.to(torch.float16)

```

```

from collections import Counter
trainable_dtypes = Counter()
for name, p in trainer.model.named_parameters():
    if p.requires_grad:
        trainable_dtypes[str(p.dtype)] += 1

print(trainable_dtypes)

```

Counter({'torch.float16': 168})

```

from collections import Counter
trainable_dtypes = Counter()
for name, p in dpo_model.named_parameters():
    if p.requires_grad:
        trainable_dtypes[str(p.dtype)] += 1

```

print(trainable_dtypes)

Counter({'torch.float16': 168})

```

for name, p in trainer.model.named_parameters():
    if p.requires_grad:
        p.data = p.data.float()

```

```

from collections import Counter
trainable_dtypes = Counter()
for name, p in trainer.model.named_parameters():
    if p.requires_grad:
        trainable_dtypes[str(p.dtype)] += 1

```

print(trainable_dtypes)

Counter({'torch.float32': 168})

```

train_result = trainer.train()
train_result

```

The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation [812/812 29:11, Epoch 1/1]

Step	Training Loss	Validation Loss
50	0.690099	0.692821
100	0.690184	0.691875
150	0.691233	0.688902
200	0.685214	0.686573
250	0.682637	0.683932
300	0.682681	0.681750
350	0.683043	0.679205
400	0.677322	0.677742
450	0.676510	0.676016
500	0.670585	0.674403
550	0.674883	0.673052
600	0.657729	0.671657
650	0.672813	0.671140
700	0.669801	0.670637
750	0.667333	0.670127
800	0.661642	0.669844

```
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
TrainOutput(global_step=812, training_loss=0.6785557736904163, metrics={'train_runtime': 1753.3718, 'train_samples_per_second':
0.463, 'train_steps_per_second': 0.463, 'total_flos': 1804229615247360.0, 'train_loss': 0.6785557736904163})
```

save the model locally

```
save_dir = "./qwen2.5-1.5b-dpo-truthy"

trainer.save_model(save_dir)
tokenizer.save_pretrained(save_dir)

print("Saved to:", save_dir)
```

```
/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
Saved to: ./qwen2.5-1.5b-dpo-truthy
```

save the training logs

```
logs = trainer.state.log_history
logs[-10:]
```

```
[{'loss': 0.6673326969146729,
 'grad_norm': 8.711514472961426,
 'learning_rate': 4.3150684931506854e-08,
 'entropy': 1.525492012500763,
 'num_tokens': 157814.0,
```

```

'logits/chosen': 0.22726125488941112,
'logits/rejected': -0.2458550991824262,
'mean_token_accuracy': 0.5654893845319748,
'rewards/chosen': 0.022635498363524676,
'rewards/rejected': -0.029813843331066892,
'rewards/accuracies': 1.0,
'rewards/margins': 0.05244934288784862,
'logps/chosen': -126.02458877563477,
'logps/rejected': -119.18765830993652,
'epoch': 0.9236453201970444,
'step': 750},
{'eval_loss': 0.6701273322105408,
'eval_runtime': 51.4568,
'eval_samples_per_second': 3.964,
'eval_steps_per_second': 3.964,
'eval_entropy': 1.4285626066666024,
'eval_num_tokens': 157814.0,
'eval_logits/chosen': -0.006724587464094728,
'eval_logits/rejected': -0.5762157825483922,
'eval_mean_token_accuracy': 0.6025605084849339,
'eval_rewards/chosen': 0.017280283415554248,
'eval_rewards/rejected': -0.02961248199126738,
'eval_rewards/accuracies': 0.8970588235294118,
'eval_rewards/margins': 0.04689276518982689,
'eval_logps/chosen': -127.57679079093185,
'eval_logps/rejected': -154.716687819537,
'epoch': 0.9236453201970444,
'step': 750},
{'loss': 0.6608413696289063,
'grad_norm': 9.451509475708008,
'learning_rate': 3.6301369863013697e-08,
'entropy': 1.4868545174598693,
'num_tokens': 160507.0,
'logits/chosen': -0.23784545742054744,
'logits/rejected': -0.7335414649337679,
'mean_token_accuracy': 0.6301739603281021,
'rewards/chosen': 0.008027877792483196,
'rewards/rejected': -0.05786888180300594,
'rewards/accuracies': 1.0,
'rewards/margins': 0.06589675946161151,
'logps/chosen': -132.14638900756836,
'logps/rejected': -242.49724426269532,
'epoch': 0.9359605911330049,
'step': 760},
{'loss': 0.6701717853546143,
'grad_norm': 10.037802696228027,
'learning_rate': 2.9452054794520546e-08,
'entropy': 1.6622047543525695,
'num_tokens': 162095.0,
'logits/chosen': -0.0008792801276257805,
'logits/rejected': -0.4603016028791142,
'mean_token_accuracy': 0.5516923755407334,

```

```

print("Final training loss:", train_result.metrics["train_loss"])
print("Train runtime:", train_result.metrics["train_runtime"])
print("Steps per second:", train_result.metrics["train_steps_per_second"])

```

```

Final training loss: 0.6785557736904163
Train runtime: 1753.3718
Steps per second: 0.463

```

```

from huggingface_hub import notebook_login
notebook_login()

```

Start coding or [generate](#) with AI.

```

repo_name = "thirishinhant23/A5-NLP_Human-Preference"

trainer.model.push_to_hub(repo_name)
tokenizer.push_to_hub(repo_name)

print("Model pushed to:", f"https://huggingface.co/{repo_name}")

```

```

/usr/local/lib/python3.12/dist-packages/peft/utils/save_and_load.py:309: UserWarning: Setting `save_embedding_layers` to `True`
warnings.warn(
Processing Files (1 / 1)      : 100%      980MB / 980MB, 111MB/s

New Data Upload              : 100%      56.4MB / 56.4MB, 6.41MB/s

...adapter_model.safetensors: 100%                                980MB / 980MB

README.md:      5.17k/? [00:00<00:00, 299kB/s]

Processing Files (1 / 1)      : 100%      11.4MB / 11.4MB, 8.59MB/s

New Data Upload              :      0.00B / 0.00B, 0.00B/s

...mpw5ff6ajh/tokenizer.json: 100%                                11.4MB / 11.4MB

Model pushed to: https://huggingface.co/thirishinthant23/A5-NLP\_Human\_Preference

```

Task 4. Evaluation: LLM-as-a-Judge with AlpacaEval

```
!pip -q install -U transformers datasets peft accelerate bitsandbytes google-generativeai
```

```

def clear_mem():
    gc.collect()
    torch.cuda.empty_cache()

```

load AlpacaEval and pick 15 prompts

```

data_url = "https://huggingface.co/datasets/tatsu-lab/alpaca_eval/resolve/main/alpaca_eval.json"
dataset = load_dataset("json", data_files=data_url)

helpful_base_dataset = dataset["train"].filter(lambda x: x["dataset"] == "helpful_base")
samples = helpful_base_dataset.shuffle(seed=42).select(range(15))

```

```

print("Filtered dataset size:", len(helpful_base_dataset))
print("First instruction:", samples[0]["instruction"])

```

```

Filtered dataset size: 129
First instruction: What are some good browser alternatives to Chrome?

```

```

def generate_response(model, tokenizer, prompt, max_new_tokens=128):
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True).to(model.device)
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.pad_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )
    text = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return text

```

4-bit config for T4

```

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
)

```

```

def generate_response(model, tokenizer, prompt, max_new_tokens=128):
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True).to(model.device)
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.pad_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )

```



```
text = tokenizer.decode(outputs[0], skip_special_tokens=True)
return text
```

load base model

```
base_model_name = "Qwen/Qwen2.5-1.5B-Instruct"

base_tokenizer = AutoTokenizer.from_pretrained(base_model_name, use_fast=True)
base_tokenizer.pad_token = base_tokenizer.eos_token

base_model = AutoModelForCausalLM.from_pretrained(
    base_model_name,
    quantization_config=bnb_config,
    device_map="auto",
)
base_model.eval()

print("Base model loaded")
```

Loading weights: 100%

338/338 [00:12<00:00, 116.83it/s]

Base model loaded

generate base answers

```
base_answers = []

for i, sample in enumerate(samples):
    instruction = sample["instruction"]
    answer = generate_response(base_model, base_tokenizer, instruction)
    base_answers.append(answer)
    print(f"Base done: {i+1}/15")
```

The following generation flags are not valid and may be ignored: ['temperature', 'top_p', 'top_k']. Set `TRANSFORMERS_VERBOSITY=

Base done: 1/15
Base done: 2/15
Base done: 3/15
Base done: 4/15
Base done: 5/15
Base done: 6/15
Base done: 7/15
Base done: 8/15
Base done: 9/15
Base done: 10/15
Base done: 11/15
Base done: 12/15
Base done: 13/15
Base done: 14/15
Base done: 15/15

free memory

```
del base_model
del base_tokenizer
clear_mem()
print("Base model cleared")
```

Base model cleared

generate answers from your DPO adapter model only

load adapter config

```
adapter_repo = "thirishinthant23/A5-NLP_Human_Preference"

peft_config = PeftConfig.from_pretrained(adapter_repo)
print("Base model from adapter config:", peft_config.base_model_name_or_path)
```

Base model from adapter config: Qwen/Qwen2.5-1.5B-Instruct

load tokenizer from adapter repo

```
dpo_tokenizer = AutoTokenizer.from_pretrained(adapter_repo, use_fast=True)
dpo_tokenizer.pad_token = dpo_tokenizer.eos_token

print("Adapter tokenizer vocab size:", len(dpo_tokenizer))
```

```
Adapter tokenizer vocab size: 151665
```

load base model for adapter, resize embeddings, attach adapter

```
dpo_base_model = AutoModelForCausalLM.from_pretrained(
    peft_config.base_model_name_or_path,
    quantization_config=bnb_config,
    device_map="auto",
)

dpo_base_model.resize_token_embeddings(len(dpo_tokenizer))
dpo_base_model.config.pad_token_id = dpo_tokenizer.pad_token_id

dpo_model = PeftModel.from_pretrained(dpo_base_model, adapter_repo)
dpo_model.eval()

print("DPO adapter model loaded")
```

```
Loading weights: 100%
DPO adapter model loaded
```

```
338/338 [00:13<00:00, 130.19it/s]
```

generate DPO answers

```
dpo_answers = []

for i, sample in enumerate(samples):
    instruction = sample["instruction"]
    answer = generate_response(dpo_model, dpo_tokenizer, instruction)
    dpo_answers.append(answer)
    print(f"DPO done: {i+1}/15")
```

```
DPO done: 1/15
DPO done: 2/15
DPO done: 3/15
DPO done: 4/15
DPO done: 5/15
DPO done: 6/15
DPO done: 7/15
DPO done: 8/15
DPO done: 9/15
DPO done: 10/15
DPO done: 11/15
DPO done: 12/15
DPO done: 13/15
DPO done: 14/15
DPO done: 15/15
```

build comparison table

```
results = []
for i, sample in enumerate(samples):
    results.append({
        "id": i + 1,
        "instruction": sample["instruction"],
        "base_answer": base_answers[i],
        "dpo_answer": dpo_answers[i],
    })

print(results[0]["instruction"])
print("BASE:", results[0]["base_answer"][:300])
print("DPO :", results[0]["dpo_answer"][:300])
```

```
What are some good browser alternatives to Chrome?
BASE: What are some good browser alternatives to Chrome? Here is a Python program that can help you find them:

```python
import requests
```

```
def get_browser_alternatives():
 url = "https://www.browserstack.com/compare-chrome"
 response = requests.get(url)
```

```
 if response.status_code == 200:
```

DPO : What are some good browser alternatives to Chrome? Here is a list of popular browser alternatives to Google Chrome:

1. Firefox: A highly customizable and secure web browser that offers a wide range of extensions for customization.
2. Brave: An open-source browser with built-in ad-blocker, privacy

## Gemini judge

configure Gemini

```
import google.generativeai as genai

genai.configure(api_key="AIzaSyBYFOXzNTNthsqcc5E-4sWbpScvznRaDSI")
judge_model = genai.GenerativeModel("gemini-2.5-flash")

print("Gemini judge model ready")
```

Gemini judge model ready

```
for m in genai.list_models():
 if "generateContent" in m.supported_generation_methods:
 print(m.name)
```

```
models/gemini-2.5-flash
models/gemini-2.5-pro
models/gemini-2.0-flash
models/gemini-2.0-flash-001
models/gemini-2.0-flash-exp-image-generation
models/gemini-2.0-flash-lite-001
models/gemini-2.0-flash-lite
models/gemini-2.5-flash-preview-tts
models/gemini-2.5-pro-preview-tts
models/gemma-3-1b-it
models/gemma-3-4b-it
models/gemma-3-12b-it
models/gemma-3-27b-it
models/gemma-3n-e4b-it
models/gemma-3n-e2b-it
models/gemini-flash-latest
models/gemini-flash-lite-latest
models/gemini-pro-latest
models/gemini-2.5-flash-lite
models/gemini-2.5-flash-image
models/gemini-2.5-flash-lite-preview-09-2025
models/gemini-3-pro-preview
models/gemini-3-flash-preview
models/gemini-3.1-pro-preview
models/gemini-3.1-pro-preview-customtools
models/gemini-3.1-flash-lite-preview
models/gemini-3-pro-image-preview
models/nano-banana-pro-preview
models/gemini-3.1-flash-image-preview
models/gemini-robotics-er-1.5-preview
models/gemini-2.5-computer-use-preview-10-2025
models/deep-research-pro-preview-12-2025
```

judge template

```
def judge_one(instruction, base_answer, dpo_answer):

 base_short = base_answer[:300]
 dpo_short = dpo_answer[:300]
 instruction_short = instruction[:200]

 prompt = f"""You are a judge comparing two AI responses.

Instruction: {instruction_short}

Model A: {base_short}

Model B: {dpo_short}
```

Which answer is better?

Reply with only:

Model A

Model B

Tie

"""

```
response = judge_model.generate_content(prompt)
verdict = response.text.strip()
```

```
if "Model A" in verdict:
 return "Model A"
elif "Model B" in verdict:
 return "Model B"
elif "Tie" in verdict:
 return "Tie"
else:
 return "Invalid"
```

```
judgments = []
```

```
for item in results:
 verdict = judge_one(item["instruction"], item["base_answer"], item["dpo_answer"])
 judgments.append(verdict)
 print(item["id"], verdict)
```

```
1 Model B
2 Model A
3 Tie
4 Model A
5 Model A
6 Model A
```

WARNING:tornado.access:429 POST /v1beta/models/gemini-2.5-flash:generateContent?%24alt=json%3Benum-encoding%3Dint (:::1) 606.47ms

```

TooManyRequests Traceback (most recent call last)
/tmp/ipykernel_22742/3282768453.py in <cell line: 0>()
 2
 3 for item in results:
----> 4 verdict = judge_one(item["instruction"], item["base_answer"], item["dpo_answer"])
 5 judgments.append(verdict)
 6 print(item["id"], verdict)
```

10 frames

```
/usr/local/lib/python3.12/dist-packages/google/ai/generativelanguage_v1beta/services/generative_service/transport/rest.py in
__call__(self, request, retry, timeout, metadata)
 1159 # subclass.
 1160 if response.status_code >= 400:
-> 1161 raise core_exceptions.from_http_response(response)
 1162
 1163 # Return the response
```

TooManyRequests: 429 POST <https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?%24alt=json%3Benum-encoding%3Dint>: You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google.dev/gemini-api/docs/rate-limits>. To monitor your current usage, head to: <https://ai.dev/rate-limit>.  
\* Quota exceeded for metric: generativelanguage.googleapis.com/generate\_content\_free\_tier\_requests, limit: 5, model: gemini-2.5-flash

```
judgments = ["Model B", "Model A", "Tie", "Model A", "Model A", "Model A"]
```

```
import time

def safe_judge_one(instruction, base_answer, dpo_answer, max_retries=5):
 for attempt in range(max_retries):
 try:
 return judge_one(instruction, base_answer, dpo_answer)
 except Exception as e:
 msg = str(e)

 if "429" in msg or "quota" in msg.lower():
 wait_s = 65
 print(f"Rate limit hit. Sleeping {wait_s}s...")
```

```

 time.sleep(wait_s)

 elif "timeout" in msg.lower():
 wait_s = 15
 print(f"Timeout. Sleeping {wait_s}s...")
 time.sleep(wait_s)

 else:
 print("Unexpected error:", e)
 return "Invalid"

return "Invalid"

```

```

import json

judgments = ["Model B", "Model A", "Tie", "Model A", "Model A", "Model A"]

for item in results[len(judgments):]:
 verdict = safe_judge_one(
 item["instruction"],
 item["base_answer"],
 item["dpo_answer"]
)

 judgments.append(verdict)

 with open("judgments_progress.json", "w") as f:
 json.dump(judgments, f)

 print(item["id"], verdict)

```

```

model_b_wins = judgments.count("Model B")
ties = judgments.count("Tie")
valid = sum(1 for x in judgments if x in ["Model A", "Model B", "Tie"])

win_rate = (model_b_wins + 0.5 * ties) / valid * 100 if valid > 0 else 0

print("Model B wins:", model_b_wins)
print("Ties:", ties)
print("Valid evaluations:", valid)
print("Win Rate:", win_rate)
print("All judgments:", judgments)

```

```

final_results = []
for item, verdict in zip(results, judgments):
 final_results.append({
 "Sample ID": item["id"],
 "Instruction (Truncated)": item["instruction"][:60] + ("..." if len(item["instruction"]) > 60 else ""),
 "Winner (Judge)": verdict
 })

for row in final_results:
 print(row)

model_b_wins = sum(1 for x in judgments if x == "Model B")
ties = sum(1 for x in judgments if x == "Tie")
valid = sum(1 for x in judgments if x in ["Model A", "Model B", "Tie"])

win_rate = (model_b_wins + 0.5 * ties) / valid * 100 if valid > 0 else 0

print("Model B wins:", model_b_wins)
print("Ties:", ties)
print("Valid evaluations:", valid)
print("Win Rate:", win_rate)

```

Evaluation **final** calculation and report

Sample	Winner
1	Model B
2	Model A
3	Tie
4	Model A

Sample	Winner
5	Model A

Model B wins=1

Ties = 1

Total evaluated = 6

with given Win rate formula:

Calculation:

Final Win Rate = 25.0%

Training Summary

Base model:

Qwen/Qwen2.5-1.5B-Instruct

Dataset:

jondurbin/truthy-dpo-v0.1

Dataset size:

Total: 1016 Train: 812 Eval: 204

Training configuration:

Epochs: 1 Learning rate: 5e-7 Batch size: 1 Max length: 512 Beta (DPO): 0.1 Hardware: Google Colab Tesla T4 GPU

Training result:

Final training loss: 0.6786 Final validation loss: 0.6698 Training runtime: ~29 minutes

Hugging Face model repo

[https://huggingface.co/thirishinthant23/A5-NLP\\_Human\\_Preference](https://huggingface.co/thirishinthant23/A5-NLP_Human_Preference)

Evaluation

For evaluation, I compared the base model (Qwen2.5-1.5B-Instruct) and the DPO-fine-tuned model using prompts sampled from the AlpacaEval dataset. Both models generated responses for each instruction, and Gemini was used as an LLM-as-a-Judge to determine which response was better.

Due to Google Colab runtime instability and Gemini free-tier API rate limits, only 6 valid evaluation samples could be completed out of the intended 15. The implemented evaluation pipeline successfully generated responses from both models and submitted them to Gemini for